postlight.course / v10

> bootcamp.learn()

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 20

JavaScript collections and immutability

learn / type / run / explain

Plain meaning and technical view

PLAIN MEANING

UI records are objects inside arrays. A change should create new state rather than mutate old state.

TECHNICAL VIEW

Destructuring, optional access, map/filter/find and spread express collection reads and immutable updates.

REAL-WORLD USE

Search, edit and remove posts by stable ID before introducing React state.

QUICK TIP

map creates a new array but does not automatically clone nested objects. Use stable IDs, not display positions.

INSTALL / RUN

No new package. Run node FILE.js.

Worked example 1: Read object fields

chapter_20_javascript_collections_and_immutability.js

```
01  const post = { id: 1, title: "Hello", image: null };
02  const { title } = post;
03  console.log(title, post.author?.email ?? "Unknown");
```

PREDICT When does ?? choose its fallback?

Run and explain example 1

EXPECTED OUTPUT

Hello Unknown

WHY IT BEHAVES THIS WAY

Only for null or undefined. Optional chaining avoids dereferencing a missing value.

Worked example 2: Map, filter and find

chapter_20_javascript_collections_and_immutability.js

```
01  const posts = [{ id: 1, title: "Python" }, { id: 2, title: "CSS" }];
02  console.log(posts.map((p) => p.title));
03  console.log(posts.filter((p) => p.title.toLowerCase().includes("py")).length);
04  console.log(posts.find((p) => p.id === 99));
```

PREDICT Does find return an array?

Run and explain example 2

EXPECTED OUTPUT

Titles Python,CSS. Filter length 1. Missing find gives undefined.

WHY IT BEHAVES THIS WAY

find returns the first matching element or undefined. filter returns an array.

Exercise 1: Case-insensitive search

YOUR TASK

Return matching post IDs using a trimmed query. Leave source records unchanged.

CHECK THESE CASES

" PY " returns ID 1. Empty query returns all records.

HINT AFTER A TRY

Normalize once, then filter before map.

Worked example 3: Spread copies

chapter_20_javascript_collections_and_immutability.js

```
01  const original = { id: 1, title: "Old" };
02  const draft = { ...original, title: "New" };
03  const posts = [original];
04  const next = [...posts, { id: 2, title: "Added" }];
05  console.log(original.title, draft.title, posts.length, next.length);
```

PREDICT Are spread copies deep copies?

Run and explain example 3

EXPECTED OUTPUT

Old New 1 2

WHY IT BEHAVES THIS WAY

No. Nested references can still be shared.

Worked example 4: Update and delete by ID

chapter_20_javascript_collections_and_immutability.js

```
01  const posts = [{ id: 1, title: "A" }, { id: 2, title: "B" }];
02  const edited = posts.map((p) => p.id === 2 ? { ...p, title: "New" } : p);
03  const remaining = edited.filter((p) => p.id !== 1);
04  console.log(posts[1].title, remaining[0].title);
```

PREDICT Why is an ID safer than the array index?

Run and explain example 4

EXPECTED OUTPUT

B New

WHY IT BEHAVES THIS WAY

Filtering and ordering change positions. IDs identify the same record across views.

Exercise 2: Immutable edit/remove

YOUR TASK

Change ID 2 title then remove ID 1. Verify the original array and record are unchanged.

CHECK THESE CASES

Source titles A,B. Final ID 2 title C.

HINT AFTER A TRY

Never assign posts[index].title while treating posts as React state.

Checkpoint before moving on

SHOW AND EXPLAIN

Run all four examples, complete both tasks, and explain search, edit and remove posts by stable id before introducing react state.

TRANSFER TO POSTLIGHT

Search, edit and remove posts by stable ID before introducing React state.